JAVA PROGRAMMING

Chapter 16: Inheritance in Java

Personal Notes

Inheritance is one of the four pillars of OOP. It lets a class reuse the properties and behaviors of another class — instead of writing everything from scratch.

In real projects, inheritance is what stops you from copy-pasting the same fields and methods into dozens of related classes. Done well, it gives you cleaner code, less duplication, and a clear class hierarchy.

1. What is Inheritance?

Inheritance is a mechanism where one class (called the CHILD class) acquires the properties and methods of another class (called the PARENT class).

Terminology

- Parent class — also called Superclass or Base class. The class being inherited FROM.
- Child class — also called Subclass or Derived class. The class doing the inheriting.
- extends — the Java keyword used to set up inheritance.

The IS-A Relationship

Inheritance represents an "IS-A" relationship. For example, a Dog IS-AN Animal. A Car IS-A Vehicle. A Student IS-A Person. This is the mental test you can use to decide whether inheritance fits — if you can't honestly say "X is a Y," don't use inheritance.

First Example: Animal and Dog

```
// Parent class
class Animal {
    String name;

    void eat() {
        System.out.println(name + " is eating.");
    }

    void sleep() {
        System.out.println(name + " is sleeping.");
    }
}

// Child class – inherits everything from Animal
class Dog extends Animal {
    void bark() {
```

```
        System.out.println(name + " says: Woof!");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.name = "Bruno";
        d.eat();    // inherited from Animal
        d.sleep();  // inherited from Animal
        d.bark();   // defined in Dog
    }
}
```

Visual

```
          Animal              (parent)
         +---------+
         | name    |
         | eat()   |
         | sleep() |
         +---------+
              |
              | extends
              v
         +---------+
         | bark()  |            (child — also gets
         +---------+             name, eat(), sleep())
           Dog
```

2. Types of Inheritance in Java

Java supports three types of inheritance with classes: Single, Multilevel, and Hierarchical.

2.1 Single Inheritance

One parent class and one child class. This is the simplest form.

```
     Animal              (parent)
        |
        v
       Dog                (child)
```

```
class Animal {
    void eat() {
        System.out.println("This animal eats food.");
    }
}

class Dog extends Animal {
```

```
    void bark() {
        System.out.println("Dog says: Woof!");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();    // inherited
        d.bark();   // own method
    }
}
```

2.2 Multilevel Inheritance

A class inherits from another derived class, forming a chain. Grandchild → Child → Parent.

```
     Animal              (grandparent)
        |
        v
       Dog                (parent)
        |
        v
     Puppy                (child)
```

```
class Animal {
    void eat() {
        System.out.println("This animal eats food.");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks.");
    }
}

class Puppy extends Dog {
    void weep() {
        System.out.println("Puppy weeps.");
    }
}

public class Main {
    public static void main(String[] args) {
        Puppy p = new Puppy();
        p.eat();    // from Animal
        p.bark();   // from Dog
        p.weep();   // own
    }
```

```
}
```

Puppy inherits from Dog, which inherits from Animal. So Puppy gets everything from BOTH ancestors.

2.3 Hierarchical Inheritance

Multiple child classes inherit from the SAME parent class. Like siblings sharing a parent.

```
              Animal              (parent)
              /     \
             /       \
            v         v
          Dog         Cat          (siblings — both extend Animal)
```

```
class Animal {
    void eat() {
        System.out.println("Animal is eating.");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks.");
    }
}

class Cat extends Animal {
    void meow() {
        System.out.println("Cat meows.");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();    // inherited
        d.bark();   // own

        Cat c = new Cat();
        c.eat();    // inherited
        c.meow();   // own
    }
}
```

2.4 What About Multiple Inheritance?

Java does NOT support multiple inheritance with classes — a class cannot extend more than one parent class. This is intentional, to avoid the Diamond Problem.

The Diamond Problem

```
          A              (grandparent)
         / \             both B and C have a method
        v   v             called show()
        B   C
         \ /
          v v
           D              if D inherits both, which show() runs?
                          → AMBIGUITY → Java refuses to allow this
```

Workaround: Java DOES allow multiple inheritance through INTERFACES. A class can implement many interfaces, since interfaces traditionally have no implementation, so there's no ambiguity.

2.5 Quick Comparison

Type	Description	Example
Single	One child, one parent	Dog extends Animal
Multilevel	Inheritance in a chain	Puppy → Dog → Animal
Hierarchical	Many children, one parent	Dog, Cat both extend Animal
Multiple	NOT allowed with classes (use interfaces)	Dog extends Animal, Pet — invalid

3. Access Specifiers in Inheritance

Access modifiers decide which members are visible inside the child class. This matters a lot when designing parent classes.

Modifier	Where accessible	In subclass?
public	Everywhere	Yes — inherited and accessible
protected	Same package + subclasses	Yes — inherited and accessible
default	Same package only (no keyword)	Only if same package
private	Same class only	Inherited but NOT directly accessible

Example

```
class Parent {
    public    int   a = 1;     // visible everywhere
    protected int   b = 2;     // visible to subclasses (even other
packages)
              int   c = 3;     // default — same package only
    private   int   d = 4;     // NOT visible to children directly
}

class Child extends Parent {
    void show() {
        System.out.println(a);    // OK — public
        System.out.println(b);    // OK — protected
        System.out.println(c);    // OK if same package
        // System.out.println(d); // ERROR — private not visible
    }
}
```

> **Rule of thumb:** Use protected when you want subclasses to access something but not the whole world. Use private + a getter when you want full encapsulation even in subclasses.

4. The super Keyword

The super keyword is used inside a child class to refer to its parent class. It has two main uses.

4.1 super() — Call Parent Constructor

Use super(...) on the FIRST line of a child constructor to call the parent's constructor.

```
class Animal {
    String name;

    Animal(String name) {
        this.name = name;
        System.out.println("Animal constructor called");
    }
}

class Dog extends Animal {
    String breed;

    Dog(String name, String breed) {
        super(name);              // calls Animal's constructor
        this.breed = breed;
        System.out.println("Dog constructor called");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Dog d = new Dog("Bruno", "Labrador");
    }
}
```

Output:

```
Animal constructor called
Dog constructor called
```

4.2 super.methodName() — Call Parent Method

When a child overrides a parent method but still wants to use the parent version, super.methodName() lets you call it.

```
class Animal {
    void sound() {
        System.out.println("Some generic animal sound");
    }
}

class Dog extends Animal {
    void sound() {
        super.sound();                          // call parent version
first
        System.out.println("Dog says: Woof!");  // then add child behavior
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound();
    }
}
```

Output:

```
Some generic animal sound
Dog says: Woof!
```

5. Constructors in Inheritance

Constructors are NOT inherited in Java. But when a child object is created, the parent's constructor still runs FIRST — automatically — to make sure the parent part of the object is properly initialized.

Automatic Behavior

If you don't write super(...) yourself, Java silently inserts super() (the no-arg version) as the first line of every child constructor. This means the parent's no-arg constructor MUST exist, or you must call a specific one explicitly.

Example

```
class A {
    A() {
        System.out.println("A constructor");
    }
}

class B extends A {
    B() {
        // Java inserts: super();  <-- automatically
        System.out.println("B constructor");
    }
}

class C extends B {
    C() {
        // Java inserts: super();  <-- automatically
        System.out.println("C constructor");
    }
}

public class Main {
    public static void main(String[] args) {
        C c = new C();
    }
}
```

Output:

```
A constructor
B constructor
C constructor
```

Execution Order: Constructors always run from the TOP of the inheritance chain downward — grandparent, then parent, then child. This guarantees the parent's state is set up before any child code touches it.

Important: If your parent class only has a parameterized constructor (no no-arg), the child MUST explicitly call super(...) with arguments. Otherwise the code won't compile because the automatic super() call won't find a matching constructor.

6. Full Example: Vehicle System

Here is a realistic inheritance example showing how it's used in practice.

```
// Parent class
class Vehicle {
    String brand;
    int speed;

    Vehicle(String brand, int speed) {
        this.brand = brand;
        this.speed = speed;
    }

    void start() {
        System.out.println(brand + " is starting...");
    }

    void stop() {
        System.out.println(brand + " has stopped.");
    }
}

// Child class 1
class Car extends Vehicle {
    Car(String brand, int speed) {
        super(brand, speed);
    }

    void honk() {
        System.out.println(brand + " honks: Beep beep!");
    }
}

// Child class 2
class Truck extends Vehicle {
    int loadCapacity;

    Truck(String brand, int speed, int loadCapacity) {
        super(brand, speed);
        this.loadCapacity = loadCapacity;
    }

    void loadCargo() {
        System.out.println(brand + " is loading " + loadCapacity + " kg");
    }
}

public class Main {
```

```
    public static void main(String[] args) {
        Car c = new Car("Toyota", 180);
        c.start();    // inherited
        c.honk();     // own
        c.stop();     // inherited

        Truck t = new Truck("Volvo", 100, 5000);
        t.start();      // inherited
        t.loadCargo();  // own
        t.stop();       // inherited
    }
}
```

Both Car and Truck inherit start() and stop() from Vehicle. Each adds its own specific behavior. No code is duplicated — that's the win.

7. Real-World Analogy

Think of a Vehicle as a parent class. A Car, Bike, and Truck can all inherit common features like starting and stopping from the Vehicle class.

```
              Vehicle
        (start, stop, brand)
            /     |     \
           /      |      \
          v       v       v
        Car     Bike     Truck
       (honk) (kickstand)(loadCargo)
```

Each vehicle can also have its own specific features:

- Car has a honk() method
- Bike has a kickStand() method
- Truck has a loadCargo() method and load capacity

Common functionality is REUSED, and unique behavior is ADDED where needed. That is exactly how inheritance is used in real projects.

8. Advantages of Inheritance

- Code Reusability — write logic once, use it across many classes
- Reduced Redundancy — no copy-pasting shared fields and methods
- Supports Polymorphism — child classes can override parent methods
- Better Organization — creates a logical class hierarchy
- Easier Maintenance — fix or improve the parent and all children benefit
- Models Real-World IS-A relationships clearly

9. The final Keyword in Inheritance

The final keyword has special meaning in inheritance. It can be applied to classes, methods, and variables.

final class — cannot be extended

```
final class Calculator {
    // ...
}

class MyCalc extends Calculator {   // ERROR — Calculator is final
}
```

Useful when you want to lock down a class — like Java's built-in String class, which is final.

final method — cannot be overridden

```
class Parent {
    final void show() {
        System.out.println("Locked!");
    }
}

class Child extends Parent {
    void show() {   // ERROR — show() is final in parent
    }
}
```

final variable — cannot be reassigned

```
final int MAX = 100;
MAX = 200;   // ERROR
```

10. Common Mistakes

Mistake 1: Using inheritance for code reuse only

Don't extend a class just because you want some of its methods. Use inheritance ONLY when the IS-A relationship is true. If Dog isn't really an Animal, don't inherit — use composition (have-a relationship) instead.

Mistake 2: Trying to inherit from multiple classes

```
class Dog extends Animal, Pet { ... }   // ERROR — multiple inheritance
not allowed
```

Use interfaces instead: class Dog extends Animal implements Pet { ... }

Mistake 3: Forgetting super() when parent has no no-arg constructor

```
class Parent {
    Parent(String s) { ... }   // only parameterized
}

class Child extends Parent {
    Child() {                  // ERROR — implicit super() finds no match
    }
}
```

Fix: explicitly call super("value") in Child's constructor.

Mistake 4: Trying to access private parent fields directly

Private fields ARE inherited but cannot be touched directly in the child. Use getters/setters or change the modifier to protected.

Mistake 5: Overusing inheritance

Inheritance creates tight coupling between classes. Deep hierarchies become hard to maintain. Modern best practice: prefer composition over inheritance when in doubt.

11. Quick Reference

Term / Keyword	Meaning
extends	Keyword to inherit from a class
super()	Calls the parent class constructor
super.method()	Calls a parent method from inside the child
IS-A	The relationship inheritance represents
Single	One parent, one child
Multilevel	Inheritance chain (A → B → C)
Hierarchical	Many children share one parent
Multiple (not allowed)	Classes can't have two parents — use interfaces
Diamond Problem	Conflict from two parents with same method
final class	Cannot be extended
final method	Cannot be overridden

12. Practice Problems

Try these on your own to lock in the concepts.

1. Create a Person class with name and age. Build a Student class that extends Person and adds rollNumber.
2. Design a hierarchical inheritance: Shape as parent, with Circle, Rectangle, and Triangle as children. Each implements an area() method.
3. Create a multilevel chain: Animal → Mammal → Human. Each level adds its own field and method.
4. Build an Employee class with calculateSalary(). Create a Manager subclass that overrides it to add a bonus, using super.calculateSalary().
5. Create a BankAccount parent with balance, deposit, withdraw. Build SavingsAccount and CurrentAccount as children with different rules.
6. Make a final class and try to extend it — observe the compiler error.
7. Build a Vehicle class with a parameterized constructor and a Car subclass. Use super(...) correctly.
8. Create a 4-level inheritance chain and print the constructor execution order.

13. Points to Remember

- Use the extends keyword to inherit
- Java supports single, multilevel, and hierarchical inheritance through classes
- Multiple inheritance with classes is NOT allowed (Diamond Problem) — use interfaces
- Constructors are NOT inherited, but the parent constructor runs FIRST via implicit super()
- super refers to the parent class — use super() for constructors, super.x for fields/methods
- A class declared as final cannot be inherited
- Inheritance models the IS-A relationship — don't force it where it doesn't fit

14. Final Takeaway

Inheritance is one of the most powerful tools in OOP — when used wisely. It eliminates duplicate code, creates clean hierarchies, and sets the stage for polymorphism.

But it is also one of the easiest things to overuse. Deep inheritance chains and forced parent-child relationships make code rigid and hard to change. The modern rule is: use inheritance when there's a clear IS-A relationship, and reach for composition (HAS-A) when there isn't.

> **Key Takeaway:** Inheritance answers one simple question: "Can I reuse what this other class already does, and add or change a little of my own?" If yes, extend. If you're just copying methods because they're useful, that's not inheritance — that's an excuse. Composition is often the better tool.

15. What's Next?

Now that inheritance feels natural, the next logical OOP topics are:

- Polymorphism — method overriding and dynamic dispatch in depth
- Abstract classes — partial blueprints for subclasses
- Interfaces — pure abstraction and multiple inheritance
- Object class — the silent parent of every Java class
- Composition vs Inheritance — when to use which
- The instanceof keyword and type casting between parent and child